

LAPORAN PROYEK AKHIR TEKNIK KOMPILASI
PERANCANGAN DAN IMPLEMENTASI KOMPIER
SEDERHANA BAHASA MINIPASCAL MENGGUNAKAN
PYTHON



Dosen Pengajar:

Assoc. Prof. Dr. Susandri, S.Kom, M.Kom

Disusun Oleh:

Kelompok 5 :

1. MELATI SURYA NINGSIH (2455201003)
2. DIVA FILISA (2455201085)
3. MUHAMMAD RIDHO RENALDI (2455201048)
4. RUPINTA PRIAYU SITORUS (2455201024).
5. OZA SULTA WINANDA (2455201042)
6. MARGOGO ARYANO (2455201099)

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS LANCANG KUNING

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga laporan Proyek Akhir mata kuliah Teknik Kompilasi ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi dari proyek pembuatan kompiller sederhana bahasa MiniPascal menggunakan Python. Proyek yang dikerjakan meliputi implementasi Scanner, Parser, Abstract Syntax Tree (AST), Semantic Analyzer, Intermediate Code Generation (Three Address Code), Optimasi, serta Error Handling sesuai dengan materi yang telah dipelajari pada mata kuliah Teknik Kompilasi.

Penulis mengucapkan terima kasih kepada Bapak Assoc. Prof. Dr. Susandri, S.Kom., M.Kom. selaku dosen pengampu mata kuliah Teknik Kompilasi yang telah memberikan arahan dan bimbingan selama proses pengerjaan proyek ini. Terima kasih juga kepada seluruh anggota Kelompok 5 yang telah bekerja sama dalam menyelesaikan proyek dan penyusunan laporan ini.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan mengenai implementasi teknik kompilasi, khususnya dalam pengembangan kompiller sederhana.

Pekanbaru, 19 Juni 2026

Kelompok 5

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR	i
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan.....	3
1.4 Batasan Masalah.....	3
BAB II DESAIN SISTEM.....	4
2.1 Gambaran Umum Sistem	4
2.2 Arsitektur Compiler.....	6
2.3 Desain Scanner.....	7
2.4 Desain Parser dan Abstract Syntax Tree (AST)	10
2.4.1 Desain Parser.....	10
2.4.2 Desain Abstract Syntax Tree (AST).....	11
2.5 Grammar MiniPascal	12
2.6 Desain Semantic Analyzer	13
2.7 Desain Intermediate Code Generation (ICG).....	15
2.8 Desain Optimasi	17
2.8.1 Constant Folding	18
2.8.2 Dead Code Elimination	18
BAB III IMPLEMENTASI SISTEM.....	19
3.1 Lingkungan Pengembangan	19

3.2 Implementasi Scanner	19
3.3 Implementasi Parser dan AST	21
3.4 Implementasi Semantic Analyzer	22
3.5 Implementasi Intermediate Code Generation (ICG)	23
3.6 Implementasi Optimasi	23
3.7 Implementasi Program Utama.....	24
3.8 Pengujian Sistem.....	25
3.8.1 Pengujian Scanner	25
3.8.2 Pengujian Parser dan AST	25
3.8.3 Pengujian Semantic Analyzer	26
3.8.4 Pengujian Intermediate Code Generation (ICG).....	26
3.8.5 Pengujian Optimasi	27
3.8.6 Hasil Pengujian Keseluruhan	27
BAB IV PENUTUP	28
4.1 Kesimpulan	28
4.2 Saran.....	28
DAFTAR PUSTAKA	30

DAFTAR GAMBAR

Gambar 2. 1 Sistem	5
Gambar 2. 2 Arsitektur Compiler	7
Gambar 2. 3 Flowchart Desain Scanner	9
Gambar 2. 4 Flowchart Parser dan AST	10
Gambar 2. 5 Flowchart Semantic Analyzer	14
Gambar 2. 6 Flowchart Intermediate Code Generation (ICG)	16
Gambar 2. 7 Flowchart Optimasi	17
Gambar 3. 1 file utama	19
Gambar 3. 2 Implementasi Scanner	20
Gambar 3. 3 Implementasi Scanner (2)	20
Gambar 3. 4 Implementasi Scanner	21
Gambar 3. 5 Implementasi Parser dan AST	21
Gambar 3. 6 Implementasi Semantic Analyzer	22
Gambar 3. 7 Implementasi Semantic Analyzer(2)	22
Gambar 3. 8 Implementasi Intermediate Code Generation (ICG)	23
Gambar 3. 9 Implementasi Optimasi	23
Gambar 3. 10 Implementasi Program Utama	24
Gambar 3. 11 Implementasi Program Utama (2)	24
Gambar 3. 12 Pengujian Scanner	25
Gambar 3. 13 Pengujian Parser dan AST	25
Gambar 3. 14 Pengujian Semantic Analyzer	26
Gambar 3. 15 Pengujian Intermediate Code Generation (ICG)	26
Gambar 3. 16 Pengujian Optimasi	27

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat mendorong penggunaan berbagai bahasa pemrograman dalam pengembangan perangkat lunak. Bahasa pemrograman tingkat tinggi dirancang agar lebih mudah dipahami dan digunakan oleh manusia dalam menyusun instruksi untuk komputer. Namun, komputer tidak dapat secara langsung memahami bahasa tingkat tinggi sehingga diperlukan suatu mekanisme penerjemahan ke dalam bentuk yang dapat diproses oleh mesin.

Compiler merupakan perangkat lunak yang berfungsi untuk menerjemahkan program sumber (source code) menjadi bahasa yang dapat dipahami oleh komputer. Dalam proses kompilasi terdapat beberapa tahapan penting, yaitu analisis leksikal (lexical analysis), analisis sintaksis (syntax analysis), analisis semantik (semantic analysis), pembangkitan kode antara (intermediate code generation), optimasi kode, serta penanganan kesalahan (error handling). Setiap tahapan memiliki peran yang penting untuk memastikan program yang ditulis dapat diterjemahkan dengan benar dan menghasilkan keluaran yang sesuai.

Pembelajaran teknik kompilasi tidak hanya berfokus pada teori, tetapi juga pada implementasi setiap fase yang terdapat dalam sebuah compiler. Dengan memahami proses kerja compiler, mahasiswa dapat mengetahui bagaimana sebuah program diterjemahkan dari bahasa tingkat tinggi menjadi representasi yang dapat diproses oleh komputer. Selain itu, pemahaman terhadap teknik kompilasi juga membantu dalam mempelajari struktur bahasa pemrograman, tata bahasa (grammar), serta proses pendeteksian kesalahan pada program.

Pada proyek akhir ini digunakan bahasa MiniPascal sebagai bahasa sumber yang akan dikompilasi. MiniPascal dipilih karena memiliki struktur sintaks yang sederhana namun tetap mampu merepresentasikan konsep dasar dalam proses

kompilasi, seperti deklarasi variabel, assignment, percabangan, dan perulangan. Bahasa ini sangat sesuai digunakan sebagai media pembelajaran dalam implementasi compiler sederhana.

Berdasarkan latar belakang tersebut, dilakukan perancangan dan implementasi compiler sederhana bahasa MiniPascal menggunakan Python. Compiler yang dibangun mampu melakukan proses analisis leksikal, analisis sintaksis, pembentukan Abstract Syntax Tree (AST), analisis semantik, pembangkitan Three Address Code (TAC), optimasi kode, serta penanganan kesalahan sehingga dapat digunakan sebagai sarana pembelajaran dan pemahaman konsep teknik kompilasi secara lebih mendalam.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah dalam proyek ini adalah:

1. Bagaimana merancang dan mengimplementasikan compiler sederhana untuk bahasa MiniPascal menggunakan Python?
2. Bagaimana melakukan proses analisis leksikal (Scanner) terhadap kode sumber MiniPascal?
3. Bagaimana membangun Parser dan Abstract Syntax Tree (AST) berdasarkan grammar yang telah ditentukan?
4. Bagaimana melakukan analisis semantik untuk mendeteksi kesalahan pada program sumber?
5. Bagaimana menghasilkan Three Address Code (TAC) serta melakukan optimasi kode sederhana?
6. Bagaimana menerapkan mekanisme error handling untuk mendeteksi kesalahan leksikal, sintaksis, dan semantik?

1.3 Tujuan

1. Merancang dan mengimplementasikan compiler sederhana bahasa MiniPascal menggunakan Python.
2. Mengimplementasikan Scanner untuk menghasilkan token dari kode sumber.
3. Mengimplementasikan Parser dan Abstract Syntax Tree (AST) untuk memeriksa struktur sintaks program.
4. Mengimplementasikan Semantic Analyzer untuk melakukan pemeriksaan tipe data dan deklarasi variabel.
5. Menghasilkan Three Address Code (TAC) dan melakukan optimasi kode sederhana.
6. Mengimplementasikan Error Handling untuk mendeteksi kesalahan pada program sumber.

1.4 Batasan Masalah

1. Bahasa yang digunakan dalam proyek ini adalah MiniPascal.
2. Compiler dibangun menggunakan bahasa pemrograman Python.
3. Fitur yang didukung meliputi deklarasi variabel bertipe integer, assignment, if-then-else, while-do, dan begin-end.
4. Compiler melakukan proses Scanner, Parser, AST, Semantic Analyzer, TAC, Optimasi, dan Error Handling.
5. Compiler tidak mendukung array, procedure, function, maupun fitur lanjutan lainnya yang terdapat pada Pascal standar.
6. Code Generator tidak diimplementasikan karena bersifat opsional sesuai ketentuan proyek.

BAB II

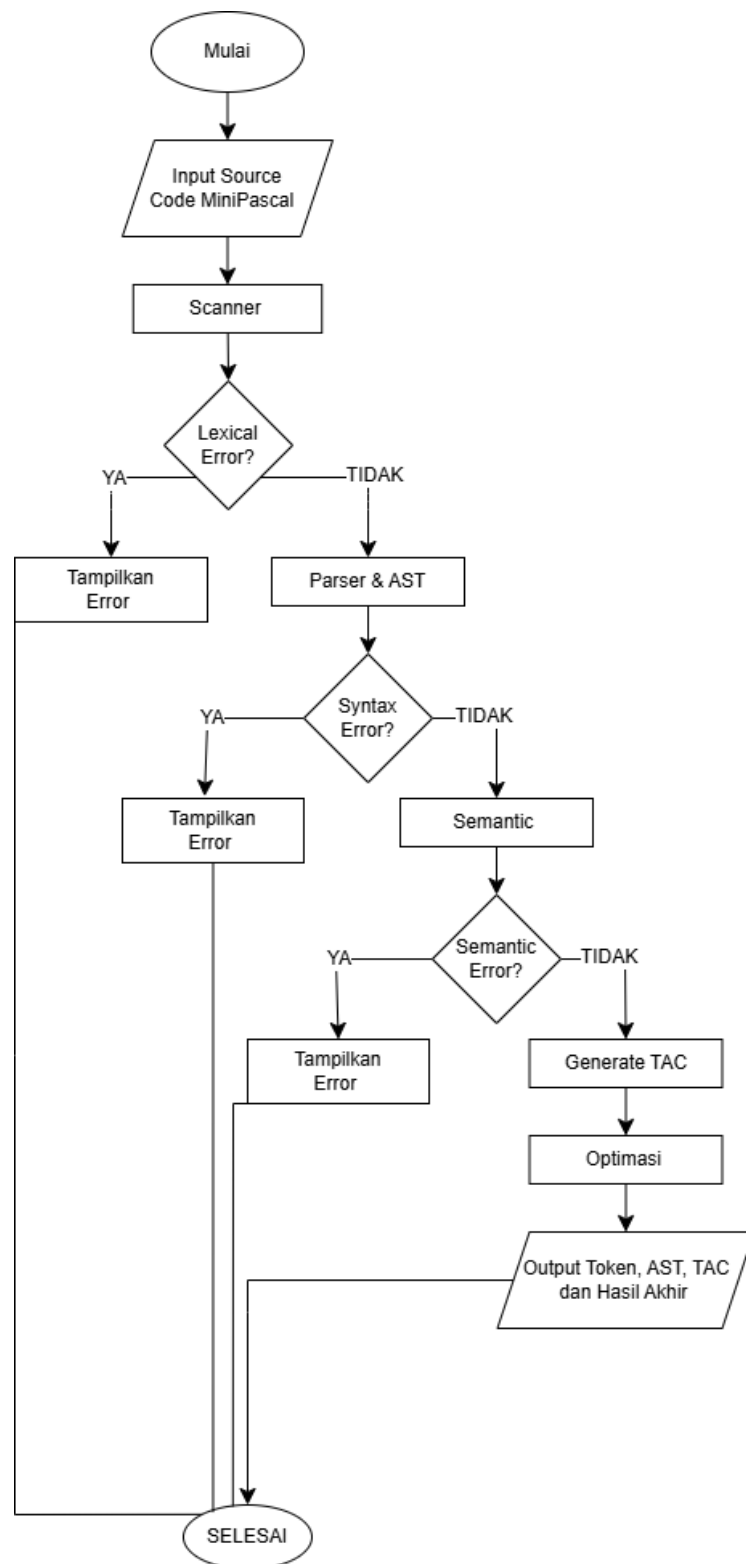
DESAIN SISTEM

2.1 Gambaran Umum Sistem

Compiler yang dikembangkan pada proyek ini merupakan compiler sederhana untuk bahasa MiniPascal yang dibangun menggunakan bahasa pemrograman Python. Compiler ini dirancang untuk melakukan beberapa tahapan proses kompilasi, mulai dari analisis leksikal, analisis sintaksis, analisis semantik, pembangkitan kode antara (Three Address Code), optimasi kode, hingga penanganan kesalahan.

Sistem menerima masukan berupa source code MiniPascal yang ditulis sesuai grammar yang telah ditentukan. Selanjutnya source code akan diproses melalui beberapa tahapan compiler secara berurutan. Hasil dari proses tersebut berupa daftar token, Abstract Syntax Tree (AST), hasil analisis semantik, Three Address Code (TAC), hasil optimasi kode, serta informasi kesalahan apabila ditemukan error pada program sumber. Compiler ini dikembangkan sebagai sarana pembelajaran untuk memahami konsep dasar teknik kompilasi serta implementasi setiap fase yang terdapat dalam sebuah compiler modern.

Untuk memberikan gambaran mengenai alur kerja sistem yang dikembangkan, berikut merupakan flowchart sistem compiler MiniPascal yang menunjukkan proses kompilasi mulai dari pembacaan source code hingga menghasilkan output akhir.



Gambar 2. 1 Sistem

Gambar 2.1 menunjukkan alur kerja compiler MiniPascal yang dimulai dari proses input source code, analisis leksikal menggunakan Scanner, analisis sintaksis menggunakan Parser dan AST, analisis semantik, pembangkitan Three Address Code (TAC), optimasi kode, hingga menghasilkan output berupa token, AST, TAC, dan hasil akhir. Pada setiap tahapan dilakukan pemeriksaan kesalahan untuk memastikan source code sesuai dengan aturan bahasa MiniPascal.

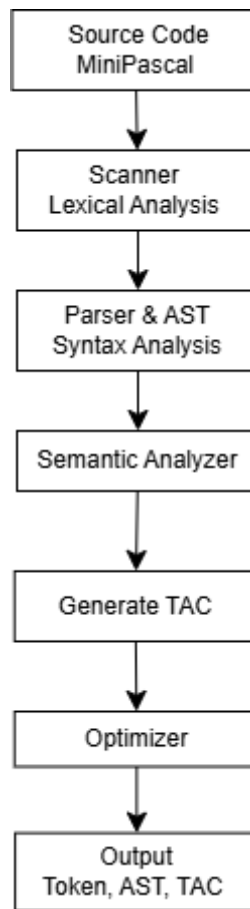
2.2 Arsitektur Compiler

Arsitektur compiler yang dikembangkan pada proyek ini terdiri dari beberapa modul yang saling terhubung dan bekerja secara berurutan dalam melakukan proses kompilasi. Setiap modul memiliki fungsi tertentu untuk mengolah source code MiniPascal hingga menghasilkan output yang diinginkan.

Proses kompilasi dimulai dari Scanner yang bertugas melakukan analisis leksikal terhadap source code dan menghasilkan token. Token yang dihasilkan kemudian diproses oleh Parser untuk melakukan pemeriksaan sintaks berdasarkan grammar MiniPascal yang telah ditentukan. Hasil dari proses parsing berupa Abstract Syntax Tree (AST) yang merepresentasikan struktur program.

AST yang telah terbentuk kemudian diproses oleh Semantic Analyzer untuk memeriksa kesesuaian penggunaan variabel, deklarasi variabel, serta tipe data yang digunakan dalam program. Setelah proses analisis semantik selesai, sistem akan menghasilkan Three Address Code (TAC) sebagai bentuk representasi kode antara.

Tahap berikutnya adalah proses optimasi kode yang bertujuan untuk meningkatkan efisiensi kode antara yang dihasilkan. Setelah seluruh tahapan selesai dilakukan, compiler akan menghasilkan output berupa token, AST, TAC, serta informasi kesalahan apabila ditemukan error selama proses kompilasi.



Gambar 2. 2 Arsitektur Compiler

Gambar 2.2 menunjukkan arsitektur compiler MiniPascal yang terdiri dari beberapa modul utama, yaitu Scanner, Parser dan AST, Semantic Analyzer, Generate TAC, serta Optimizer. Setiap modul bekerja secara berurutan untuk memproses source code hingga menghasilkan output akhir berupa token, AST, Three Address Code (TAC), dan informasi kesalahan apabila ditemukan error selama proses kompilasi.

2.3 Desain Scanner

Scanner atau lexical analyzer merupakan tahap pertama dalam proses kompilasi yang bertugas membaca source code MiniPascal dan mengubahnya menjadi kumpulan token. Token merupakan satuan terkecil yang memiliki makna dalam bahasa pemrograman, seperti keyword, identifier, operator, angka, dan simbol.

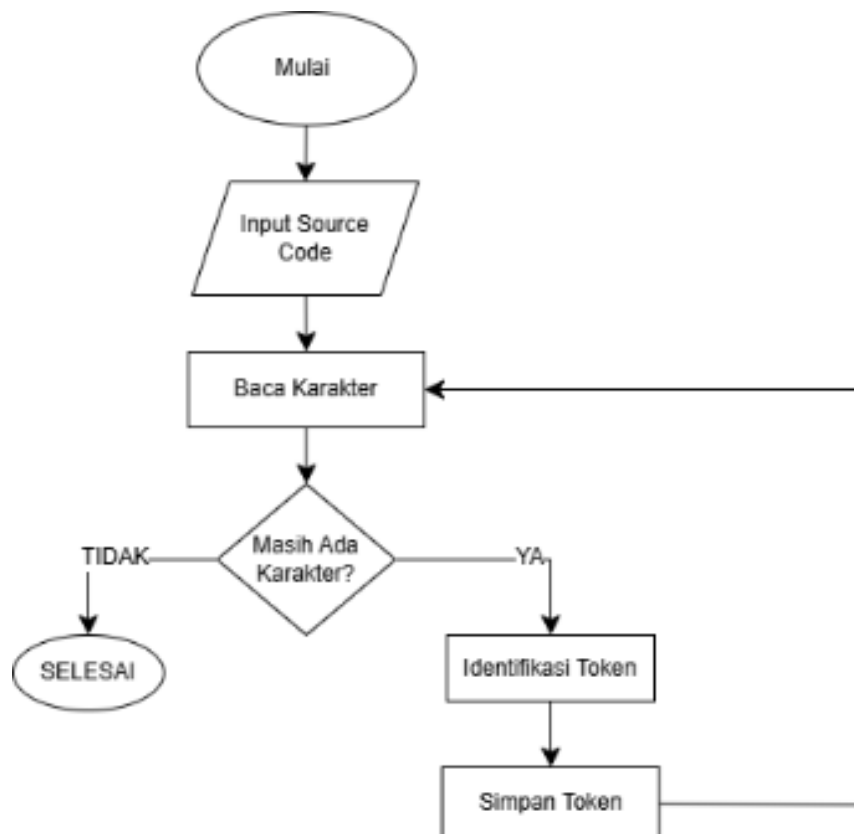
Pada compiler MiniPascal yang dikembangkan, Scanner melakukan pembacaan source code karakter demi karakter untuk mengidentifikasi jenis token yang sesuai. Hasil dari proses scanning berupa daftar token yang kemudian diteruskan ke tahap Parser untuk dilakukan analisis sintaksis.

Scanner juga berfungsi untuk mendeteksi kesalahan leksikal (lexical error), yaitu kesalahan yang terjadi apabila ditemukan karakter atau simbol yang tidak dikenali oleh bahasa MiniPascal. Dengan adanya proses analisis leksikal, compiler dapat memastikan bahwa source code yang diproses telah memenuhi aturan dasar bahasa sebelum memasuki tahap selanjutnya.

Token	Keterangan
PROGRAM	Keyword program
VAR	Keyword deklarasi variabel
INTEGER	Tipe data integer
BEGIN	Awal blok program
END	Akhir blok program
IF	Percabangan
THEN	Then
ELSE	Else
WHILE	Perulangan
DO	Do
IDENTIFIER	Nama variabel
NUMBER	Nilai numerik
ASSIGN	Operator :=
PLUS	Operator +
MINUS	Operator -
MULTIPLY	Operator *
DIVIDE	Operator /
SEMICOLON	Tanda ;

COLON	Tanda :
DOT	Tanda .

Tabel 2. 1 Desain Scanner



Gambar 2. 3 Flowchart Desain Scanner

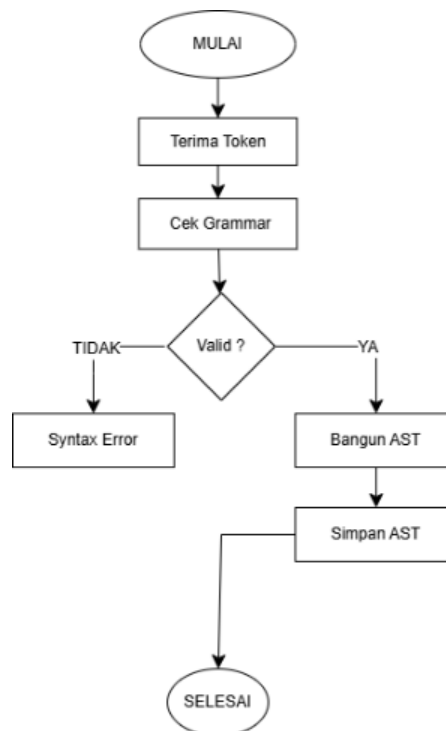
Gambar 2.3 menunjukkan alur kerja Scanner dalam melakukan analisis leksikal terhadap source code MiniPascal. Scanner membaca karakter secara berurutan, mengidentifikasi jenis token yang sesuai, kemudian menyimpan token yang berhasil dikenali. Proses ini dilakukan secara berulang hingga seluruh karakter pada source code selesai diproses. Apabila seluruh karakter telah dibaca, Scanner akan mengirimkan daftar token ke tahap Parser.

2.4 Desain Parser dan Abstract Syntax Tree (AST)

2.4.1 Desain Parser

Parser merupakan tahap kedua dalam proses kompilasi yang bertugas melakukan analisis sintaksis terhadap token yang dihasilkan oleh Scanner. Pada tahap ini, Parser akan memeriksa apakah susunan token telah sesuai dengan aturan grammar bahasa MiniPascal yang telah ditentukan.

Parser bekerja dengan membaca token secara berurutan dan mencocokkannya dengan aturan produksi (grammar). Apabila susunan token sesuai dengan grammar, maka proses parsing dinyatakan berhasil dan dilanjutkan ke pembentukan Abstract Syntax Tree (AST). Sebaliknya, apabila ditemukan ketidaksesuaian struktur sintaks, maka compiler akan menghasilkan pesan kesalahan sintaks (syntax error) dan proses kompilasi dihentikan.



Gambar 2. 4 Flowchart Parser dan AST

Gambar 2.4 menunjukkan alur kerja Parser dan Abstract Syntax Tree (AST). Parser menerima token hasil Scanner dan melakukan pemeriksaan berdasarkan grammar MiniPascal yang telah ditentukan. Apabila token tidak

sesuai dengan grammar, sistem menghasilkan syntax error dan proses kompilasi dihentikan. Jika valid, Parser membangun Abstract Syntax Tree (AST) yang merepresentasikan struktur program secara hierarkis dan menyimpannya untuk digunakan pada tahap analisis semantik.

2.4.2 Desain Abstract Syntax Tree (AST)

Abstract Syntax Tree (AST) merupakan representasi struktur program dalam bentuk pohon yang dihasilkan oleh Parser. AST digunakan untuk menggambarkan hubungan antar elemen program tanpa memperhatikan detail sintaks yang tidak diperlukan.

AST menjadi dasar dalam proses analisis semantik, pembangkitan kode antara (Three Address Code), serta optimasi kode. Dengan menggunakan AST, struktur program dapat diproses secara lebih efisien dibandingkan menggunakan source code secara langsung.

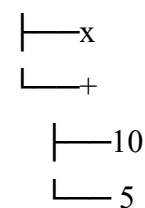
Contoh representasi AST dapat dilihat pada source code berikut :

Source Code

```
x := 10 + 5;
```

Representasi AST

Assignment



Penjelasan :

Pada contoh tersebut, node utama adalah *Assignment* yang menunjukkan proses pemberian nilai ke variabel *x*. Operasi penjumlahan direpresentasikan sebagai node operator (+) yang memiliki dua child node berupa nilai 10 dan 5. Struktur ini memudahkan compiler dalam melakukan analisis semantik, pembangkitan kode antara, serta proses optimasi kode.

2.5 Grammar MiniPascal

Grammar merupakan sekumpulan aturan yang digunakan untuk menentukan struktur sintaks suatu bahasa pemrograman. Pada compiler MiniPascal yang dikembangkan, grammar digunakan oleh Parser untuk memeriksa apakah susunan token yang diterima dari Scanner telah sesuai dengan aturan bahasa yang telah ditentukan.

Grammar MiniPascal pada proyek ini dirancang untuk mendukung beberapa fitur dasar, seperti deklarasi variabel, assignment, percabangan (*if-then-else*), perulangan (*while-do*), serta operasi aritmatika. Grammar tersebut menjadi dasar dalam proses pembentukan Abstract Syntax Tree (AST) dan analisis sintaksis yang dilakukan oleh Parser.

Berikut merupakan aturan grammar MiniPascal yang digunakan pada compiler ini.

Tabel 2.2 Grammar MiniPascal

Non-Terminal	Produksi
program	PROGRAM ID ';' declarations compound_stmt '.'
declarations	VAR decl_list
decl_list	ID ':' INTEGER ';'
compound_stmt	BEGIN stmt_list END
stmt_list	statement statement ';' stmt_list
statement	assignment if_stmt while_stmt
assignment	ID ':=' expr
if_stmt	IF condition THEN compound_stmt IF condition THEN compound_stmt ELSE compound_stmt
while_stmt	WHILE condition DO compound_stmt
expr	term expr '+' term expr '-' term
term	factor term '*' factor term '/' factor
factor	NUMBER ID '(' expr ')'

condition	expr relop expr
relop	= < > <= >= <>

Aturan grammar tersebut digunakan untuk mendefinisikan struktur program MiniPascal yang dapat diproses oleh compiler. Simbol *program* merupakan simbol awal yang mendefinisikan keseluruhan struktur program. Selanjutnya terdapat aturan deklarasi variabel, blok program (*begin-end*), assignment, percabangan, perulangan, serta ekspresi aritmatika.

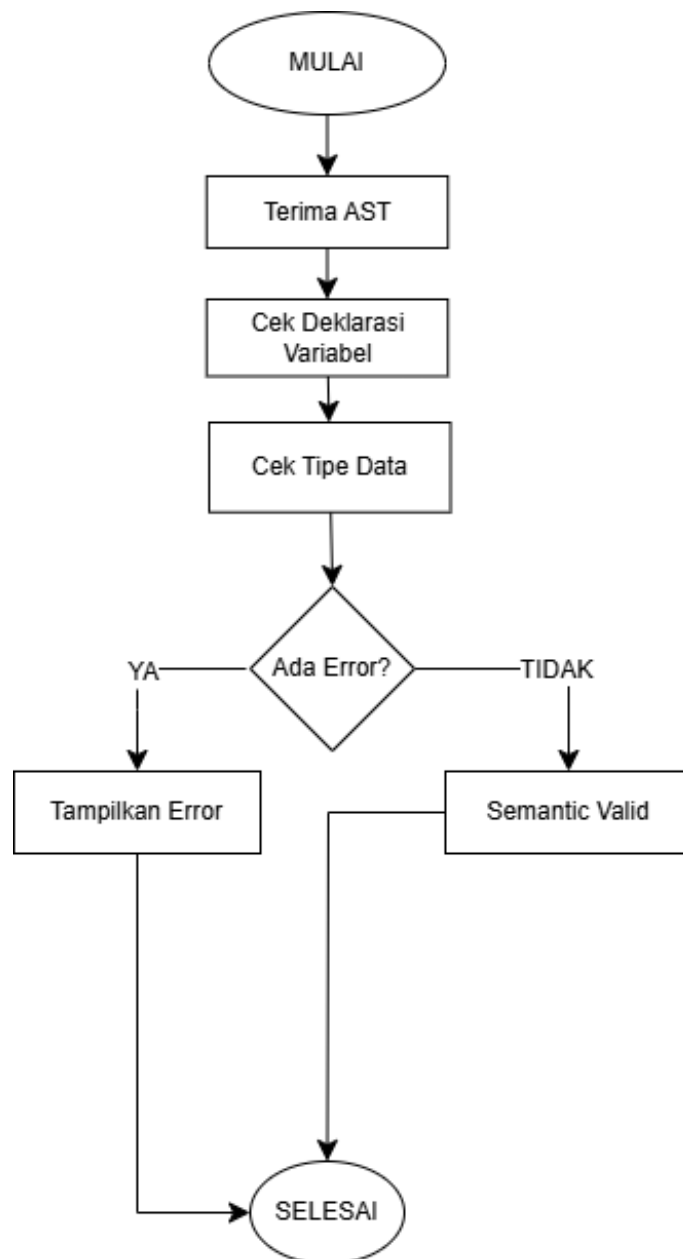
Grammar ini memungkinkan Parser untuk melakukan pemeriksaan sintaks secara sistematis dan memastikan bahwa source code yang ditulis pengguna telah sesuai dengan aturan bahasa MiniPascal. Apabila ditemukan token yang tidak sesuai dengan grammar, maka compiler akan menghasilkan pesan kesalahan sintaks (*syntax error*).

2.6 Desain Semantic Analyzer

Semantic Analyzer merupakan tahap yang bertugas memeriksa makna atau validitas program setelah berhasil melewati proses analisis sintaksis. Tahap ini menggunakan Abstract Syntax Tree (AST) yang dihasilkan oleh Parser sebagai masukan utama untuk dilakukan pemeriksaan lebih lanjut.

Pada compiler MiniPascal yang dikembangkan, Semantic Analyzer melakukan pemeriksaan terhadap deklarasi variabel, penggunaan variabel, serta kesesuaian tipe data yang digunakan dalam program. Pemeriksaan ini bertujuan untuk memastikan bahwa program tidak hanya memiliki sintaks yang benar, tetapi juga memiliki makna yang sesuai dengan aturan bahasa MiniPascal.

Apabila ditemukan kesalahan semantik, seperti penggunaan variabel yang belum dideklarasikan atau ketidaksesuaian tipe data, maka compiler akan menghasilkan pesan kesalahan dan menghentikan proses kompilasi.



Gambar 2. 5 Flowchart Semantic Analyzer

Gambar 2.5 menunjukkan alur kerja Semantic Analyzer dalam memeriksa validitas program berdasarkan AST yang dihasilkan oleh Parser. Pemeriksaan dilakukan terhadap deklarasi variabel dan kesesuaian tipe data. Apabila ditemukan kesalahan semantik, compiler akan menampilkan pesan error dan menghentikan proses kompilasi. Jika tidak ditemukan kesalahan, maka program dinyatakan valid secara semantik dan dapat dilanjutkan ke tahap pembangunan kode antara.

Contoh Semantic Error

Source Code

```
y := 10;
```

Output

Semantic Error:

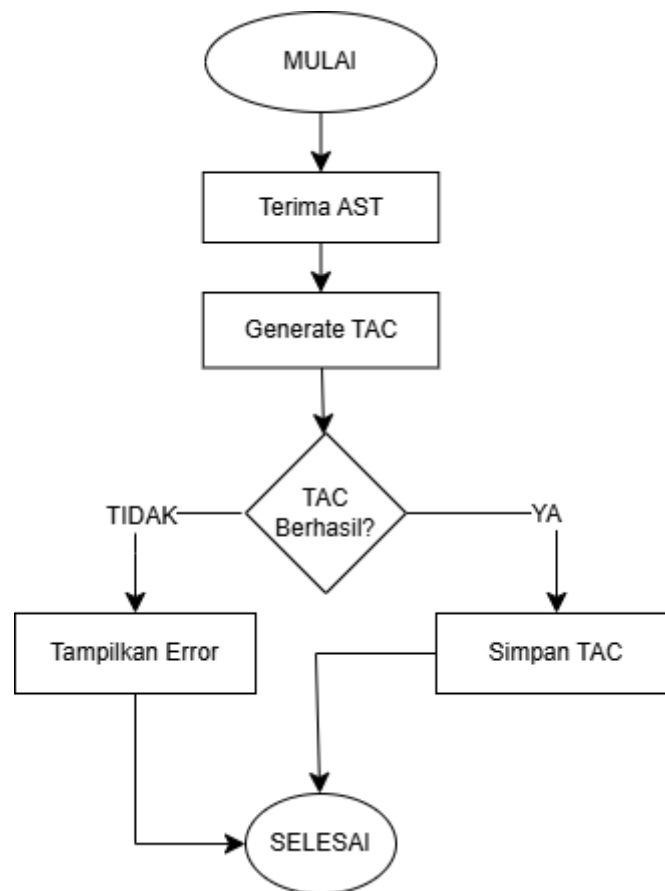
Variable 'y' not declared

2.7 Desain Intermediate Code Generation (ICG)

Intermediate Code Generation (ICG) merupakan tahap yang bertugas menghasilkan kode antara dari program yang telah lolos proses analisis semantik. Pada compiler MiniPascal yang dikembangkan, bentuk kode antara yang digunakan adalah Three Address Code (TAC).

Three Address Code (TAC) merupakan representasi program dalam bentuk instruksi sederhana yang terdiri dari operasi-operasi dasar. Setiap instruksi TAC umumnya melibatkan paling banyak tiga operand sehingga lebih mudah diproses pada tahap optimasi dan pengembangan compiler selanjutnya.

Proses pembangkitan TAC dilakukan dengan memanfaatkan struktur Abstract Syntax Tree (AST) yang telah dihasilkan sebelumnya. Setiap node pada AST diterjemahkan menjadi instruksi TAC yang merepresentasikan operasi yang dilakukan dalam program sumber.



Gambar 2. 6 Flowchart Intermediate Code Generation (ICG)

Gambar 2.6 menunjukkan proses pembangkitan Three Address Code (TAC) dari Abstract Syntax Tree (AST). Sistem menerima AST yang telah lolos proses analisis semantik, kemudian menghasilkan representasi kode antara berupa TAC. Jika proses pembangkitan TAC berhasil, hasil TAC akan disimpan untuk digunakan pada tahap optimasi. Apabila terjadi kesalahan, sistem akan menampilkan pesan error dan menghentikan proses kompilasi.

Source Code

$x := a + b * c;$

Three Address Code

$t1 = b * c$

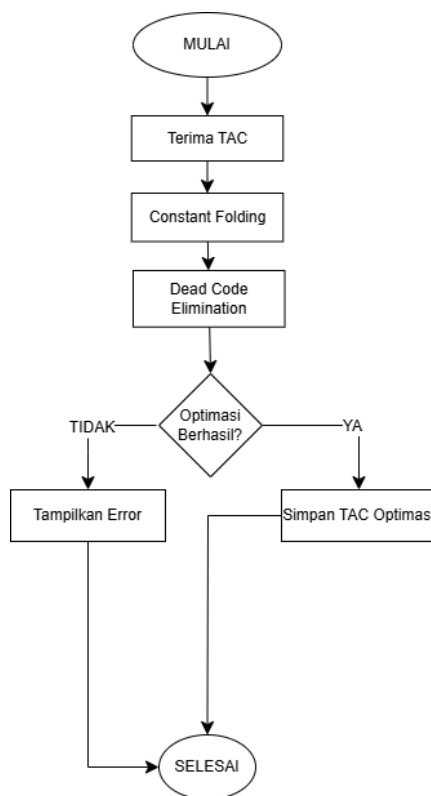
$t2 = a + t1$

$x = t2$

2.8 Desain Optimasi

Optimasi merupakan tahap yang bertujuan untuk meningkatkan efisiensi kode antara yang dihasilkan oleh compiler tanpa mengubah hasil akhir dari program. Dengan melakukan optimasi, compiler dapat mengurangi instruksi yang tidak diperlukan sehingga program menjadi lebih sederhana dan lebih efisien untuk diproses.

Pada compiler MiniPascal yang dikembangkan, digunakan dua teknik optimasi sederhana, yaitu Constant Folding dan Dead Code Elimination. Kedua teknik tersebut diterapkan pada Three Address Code (TAC) yang telah dihasilkan pada tahap sebelumnya. Optimasi dilakukan untuk menyederhanakan ekspresi yang dapat dihitung saat proses kompilasi serta menghapus kode yang tidak memberikan pengaruh terhadap hasil akhir program.



Gambar 2. 7 Flowchart Optimasi

Gambar 2.7 menunjukkan proses optimasi kode yang dilakukan setelah pembangkitan Three Address Code (TAC). Sistem menerima TAC kemudian

menerapkan teknik Constant Folding dan Dead Code Elimination untuk menyederhanakan instruksi yang ada. Apabila proses optimasi berhasil, hasil optimasi akan disimpan sebagai kode antara yang telah diperbaiki. Jika terjadi kesalahan, sistem akan menampilkan pesan error dan menghentikan proses kompilasi.

2.8.1 Constant Folding

Sebelum Optimasi

`x := 5 + 3;`

Sesudah Optimasi

`x := 8;`

Penjelasan

Constant Folding merupakan teknik optimasi yang melakukan perhitungan terhadap ekspresi konstan pada saat proses kompilasi. Dengan teknik ini, operasi yang dapat dihitung lebih awal tidak perlu dievaluasi kembali saat program dijalankan.

2.8.2 Dead Code Elimination

Sebelum Optimasi

`x:=10;`

`y := 20;`

Misalnya variabel y tidak pernah digunakan lagi.

Sesudah Optimasi

`x := 10;`

Penjelasan

Dead Code Elimination merupakan teknik optimasi yang menghapus instruksi atau variabel yang tidak digunakan dan tidak memengaruhi hasil akhir program. Teknik ini membantu mengurangi jumlah instruksi yang diproses sehingga meningkatkan efisiensi program.

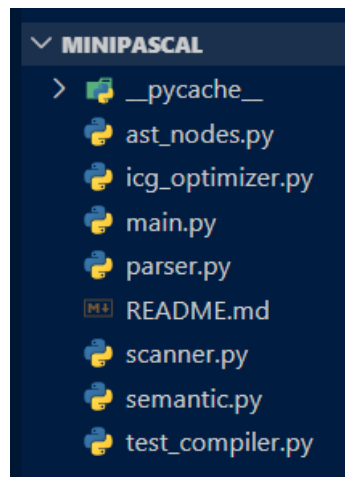
BAB III

IMPLEMENTASI SISTEM

3.1 Lingkungan Pengembangan

Compiler MiniPascal pada proyek ini dikembangkan menggunakan bahasa pemrograman Python. Proses pengembangan dilakukan dengan memanfaatkan editor Visual Studio Code sebagai lingkungan pengembangan utama. Sistem dijalankan pada sistem operasi Windows dan menggunakan pendekatan modular sehingga setiap fase compiler ditempatkan pada file yang berbeda sesuai dengan fungsinya.

Adapun file utama yang digunakan dalam implementasi compiler terdiri dari `main.py`, `scanner.py`, `parser.py`, `ast_nodes.py`, `semantic.py`, dan `icg_optimizer.py`. Setiap file memiliki tanggung jawab masing-masing dalam menjalankan proses kompilasi MiniPascal.



Gambar 3. 1 file utama

3.2 Implementasi Scanner

Scanner diimplementasikan pada file `scanner.py`. Komponen ini bertugas melakukan analisis leksikal terhadap source code MiniPascal dan mengubahnya menjadi token-token yang dapat diproses oleh Parser. Scanner mengenali berbagai jenis token seperti keyword, identifier, operator, literal, dan tanda baca.


```
# — Definisi semua tipe token —————
class TokenType:
    # Tipe data & literal
    INTEGER = 'INTEGER'
    STRING = 'STRING'
    IDENTIFIER = 'IDENTIFIER'

    # Keyword MiniPascal
    PROGRAM = 'PROGRAM'
    VAR = 'VAR'
    BEGIN = 'BEGIN'
    END = 'END'
    IF = 'IF'
    THEN = 'THEN'
    ELSE = 'ELSE'
    WHILE = 'WHILE'
    DO = 'DO'
    WRITELN = 'WRITELN'
    READLN = 'READLN'
    DIV = 'DIV'
    MOD = 'MOD'
    AND = 'AND'
    OR = 'OR'
    NOT = 'NOT'
    TRUE = 'TRUE'
    FALSE = 'FALSE'
    INTEGER_KW = 'INTEGER_KW' # keyword "integer" (tipe data)
    BOOLEAN_KW = 'BOOLEAN_KW'
    STRING_KW = 'STRING_KW'

    # Operator
    ASSIGN = 'ASSIGN' # :=
    PLUS = 'PLUS' # +
```

Gambar 3. 2 Implementasi Scanner

```
# Peta keyword → TokenType
KEYWORDS = {
    'program' : TokenType.PROGRAM,
    'var' : TokenType.VAR,
    'begin' : TokenType.BEGIN,
    'end' : TokenType.END,
    'if' : TokenType.IF,
    'then' : TokenType.THEN,
    'else' : TokenType.ELSE,
    'while' : TokenType.WHILE,
    'do' : TokenType.DO,
    'writeln' : TokenType.WRITELN,
    'readln' : TokenType.READLN,
    'div' : TokenType.DIV,
    'mod' : TokenType.MOD,
    'and' : TokenType.AND,
    'or' : TokenType.OR,
    'not' : TokenType.NOT,
    'true' : TokenType.TRUE,
    'false' : TokenType.FALSE,
    'integer' : TokenType.INTEGER_KW,
    'boolean' : TokenType.BOOLEAN_KW,
    'string' : TokenType.STRING_KW,
}
```

Gambar 3. 3 Implementasi Scanner (2)

# Operator		
ASSIGN	= 'ASSIGN'	# :=
PLUS	= 'PLUS'	# +
MINUS	= 'MINUS'	# -
MULTIPLY	= 'MULTIPLY'	# *
DIVIDE	= 'DIVIDE'	# /
EQ	= 'EQ'	# =
NEQ	= 'NEQ'	# <>
LT	= 'LT'	# <
GT	= 'GT'	# >
LTE	= 'LTE'	# <=
GTE	= 'GTE'	# >=
# Tanda baca		
SEMICOLON	= 'SEMICOLON'	# ;
COLON	= 'COLON'	# :
COMMA	= 'COMMA'	# ,
DOT	= 'DOT'	# .
LPAREN	= 'LPAREN'	# (
RPAREN	= 'RPAREN'	#)
# Spesial		
EOF	= 'EOF'	

Gambar 3. 4 Implementasi Scanner

3.3 Implementasi Parser dan AST

Parser diimplementasikan pada file parser.py dan bertugas melakukan analisis sintaksis berdasarkan grammar MiniPascal. Hasil dari proses parsing berupa Abstract Syntax Tree (AST) yang direpresentasikan menggunakan berbagai node yang didefinisikan pada file ast_nodes.py.

```
class Parser:
    def __init__(self, tokens: list):
        self.tokens = tokens
        self.pos = 0
        self.errors = []

    # helper
    def current(self) -> Token:
        return self.tokens[self.pos]

    def peek(self, offset=1) -> Token:
        idx = self.pos + offset
        if idx < len(self.tokens):
            return self.tokens[idx]
        return self.tokens[-1] # EOF

    def advance(self) -> Token:
        tok = self.tokens[self.pos]
        if tok.type != TokenType.EOF:
            self.pos += 1
        return tok

    def expect(self, ttype, msg=None) -> Token:
        tok = self.current()
        if tok.type != ttype:
            err_msg = msg or f'diharapkan {ttype} tapi ditemukan "{tok.value}" ({tok.type})'
            raise ParseError(err_msg, tok.line)
        return self.advance()

    def match(self, *ttypes) -> bool:
        return self.current().type in ttypes

    # entry point
    def parse(self) -> ProgramNode:
        try:
            node = self.parse_program()
            return node
```

Gambar 3. 5 Implementasi Parser dan AST

Gambar 3.5 menunjukkan implementasi Parser pada compiler MiniPascal. Parser menerima token yang dihasilkan oleh Scanner kemudian melakukan analisis sintaksis berdasarkan grammar yang telah ditentukan. Parser juga melakukan validasi struktur program dan menghasilkan Abstract Syntax Tree (AST) apabila proses parsing berhasil.

3.4 Implementasi Semantic Analyzer

Semantic Analyzer diimplementasikan pada file semantic.py. Komponen ini bertugas melakukan pemeriksaan deklarasi variabel, penggunaan variabel, serta kesesuaian tipe data dalam program.

```
# -- Semantic Analyzer utama --
class SemanticAnalyzer:
    def __init__(self):
        self.symbol_table = SymbolTable()
        self.errors = []

    def analyze(self, ast: ProgramNode):
        if ast is None:
            return
        try:
            self.visit_program(ast)
        except SemanticError as e:
            self.errors.append(str(e))

    # -- visitor --
    def visit_program(self, node: ProgramNode):
        # Daftarkan semua variabel ke symbol table
        for decl in node.var_decl:
            self.visit_var_decl(decl)
        # Analisis blok utama
        self.visit_block(node.block)

    def visit_var_decl(self, node: VarDeclNode):
        dtype = TYPE_MAP.get(node.type_name, node.type_name)
        for name in node.names:
            try:
                self.symbol_table.declare(name, dtype, node.line)
            except SemanticError as e:
                self.errors.append(str(e))

    def visit_block(self, node: BlockNode):
        for stmt in node.statements:
            self.visit_stmt(stmt)

    def visit_stmt(self, node):
        if node is None:
            return
```

Gambar 3. 6 Implementasi Semantic Analyzer

```
class SemanticAnalyzer:
    def visit_stmt(self, node):
        if isinstance(node, AssignNode):
            self.visit_assign(node)
        elif isinstance(node, IfNode):
            self.visit_if(node)
        elif isinstance(node, WhileNode):
            self.visit_while(node)
        elif isinstance(node, WriteNode):
            self.visit_writeln(node)
        elif isinstance(node, ReadNode):
            self.visit_readln(node)
        elif isinstance(node, BlockNode):
            self.visit_block(node)

    def visit_assign(self, node: AssignNode):
        try:
            var_type = self.symbol_table.lookup(node.var_name, node.line)
            expr_type = self.infer_type(node.expr)
            if var_type != expr_type:
                raise SemanticError(
                    f"tidak cocok tipe variabel '{node.var_name}' bertipe '{var_type}' "
                    f"tapi ekspresi bertipe '{expr_type}'",
                    node.line
                )
        except SemanticError as e:
            self.errors.append(str(e))

    def visit_if(self, node: IfNode):
        try:
            cond_type = self.infer_type(node.condition)
            if cond_type != 'boolean':
                raise SemanticError(
                    f"kondisi if harus bertipe boolean, bukan '{cond_type}'",
                    node.line
                )
        except SemanticError as e:
```

Gambar 3. 7 Implementasi Semantic Analyzer(2)

Gambar 3.6 dan 3.7 menunjukkan implementasi Semantic Analyzer yang bertugas melakukan pemeriksaan semantik terhadap AST. Semantic Analyzer memeriksa deklarasi variabel, kesesuaian tipe data, serta memastikan bahwa setiap variabel digunakan sesuai dengan tipe yang telah dideklarasikan.

3.5 Implementasi Intermediate Code Generation (ICG)

```

# -- Generator ICG
class ICGGenerator:
    def __init__(self):
        self.instructions = []
        self.temp_count = 0
        self.label_count = 0

    def new_temp(self) -> str:
        self.temp_count += 1
        return f't{self.temp_count}'

    def new_label(self) -> str:
        self.label_count += 1
        return f'l{self.label_count}'

    def emit(self, op, arg1=None, arg2=None, result=None):
        instr = TACInstruction(op, arg1, arg2, result)
        self.instructions.append(instr)
        return instr

    # -- entry
    def generate(self, ast: ProgramNode):
        self.visit_block(ast.block)
        return self.instructions

    # -- visitor
    def visit_block(self, node: BlockNode):
        for stmt in node.statements:
            self.visit_stmt(stmt)

    def visit_stmt(self, node):
        if node is None:
            return
        if isinstance(node, AssignNode):
            self.visit_assign(node)
        elif isinstance(node, IfNode):

```

Gambar 3. 8 Implementasi Intermediate Code Generation (ICG)

Gambar 3.8 menunjukkan implementasi Intermediate Code Generator (ICG) yang bertugas menghasilkan Three Address Code (TAC) dari Abstract Syntax Tree (AST). ICG menggunakan temporary variable dan label untuk merepresentasikan operasi, percabangan, serta perulangan dalam bentuk kode antara yang lebih sederhana.

3.6 Implementasi Optimasi

```

class Optimizer:
    def __init__(self, instructions: list):
        self.instructions = instructions[:]

    def optimize(self) -> list:
        self.instructions = self.constant_folding(self.instructions)
        self.instructions = self.dead_code_elimination(self.instructions)
        return self.instructions

    # -- Teknik 1: Constant Folding
    # Mengoptimalkan dengan dua operand konstanta saat compile time
    def constant_folding(self, instrs: list) -> list:
        result = []
        for instr in instrs:
            if instr.op == 'BINARY':
                left = instr.arg1
                op = instr.op
                right = instr.arg2
                lv = self.try_int(left)
                rv = self.try_int(right)
                if lv is not None and rv is not None:
                    folded = self.eval_binop(op, lv, rv)
                    if folded is not None:
                        # Ganti dengan ASSIGN langsung
                        new_instr = f'Instruction("ASSIGN", arg1=tr(folded), result=instr.result)'
                        result.append(new_instr)
                        continue
            result.append(instr)
        return result

    def try_int(self, val):
        try:
            return int(val)
        except (ValueError, TypeError):
            return None

```

Gambar 3. 9 Implementasi Optimasi

Gambar 3.9 menunjukkan implementasi Optimizer yang digunakan untuk meningkatkan efisiensi kode antara. Optimizer menerapkan teknik Constant Folding untuk menghitung ekspresi konstan saat proses kompilasi dan Dead Code Elimination untuk menghapus instruksi yang tidak digunakan.

3.7 Implementasi Program Utama

```
# --- Compile ---
def compile_minipascal(source: str, verbose: bool = True):
    print('=' * 60)
    print(' KOMPILER MINIPASCAL | Kelompok 5')
    print(' Teknik Kompilasi | Universitas Lancang Kuning')
    print('=' * 60)

    has_error = False

    # --- FASE 1: SCANNER ---
    print('\n[FASE 1] Scanning (Lexical Analysis)...')
    scanner = Scanner(source)
    tokens = scanner.tokenize()

    if scanner.errors:
        has_error = True
        print(' X Error Leksikal:')
        for err in scanner.errors:
            print(f'    → {err}')
    else:
        print(f' ✓ Berhasil menghasilkan {len(tokens)-1} token')

    if verbose:
        print('\nDaftar Token:')
        for tok in tokens:
            if tok.type != 'EOF':
                print(f'    [{tok.line:3}] {tok.type:<15} {repr(tok.value)}')

    # --- FASE 2: PARSER ---
    print('\n[FASE 2] Parsing (Syntax Analysis)...')
    parser = Parser(tokens)
    ast = parser.parse()

    if parser.errors:
        has_error = True
        print(' X Error Sintaks:')
        for err in parser.errors:
            print(f'    → {err}')
    elif ast is None:
        has_error = True
        print(' X Parsing gagal.')
    else:
        print(' ✓ AST berhasil dibangun')
```

Gambar 3. 10 Implementasi Program Utama

```
# --- FASE 2: PARSER ---
print('\n[FASE 2] Parsing (Syntax Analysis)...')
parser = Parser(tokens)
ast = parser.parse()

if parser.errors:
    has_error = True
    print(' X Error Sintaks:')
    for err in parser.errors:
        print(f'    → {err}')
elif ast is None:
    has_error = True
    print(' X Parsing gagal.')
else:
    print(' ✓ AST berhasil dibangun')

if has_error:
    print('\nX Kompilasi dihentikan karena error.')
    return False

# --- FASE 3: SEMANTIC ANALYZER ---
print('\n[FASE 3] Semantic Analysis...')
sem = SemanticAnalyzer()
sem.analyze(ast)

if sem.errors:
    has_error = True
    print(' X Error Semantik:')
    for err in sem.errors:
        print(f'    → {err}')
else:
    print(' ✓ Analisis semantik berhasil')
    sem.symbol_table.display()
```

Gambar 3. 11 Implementasi Program Utama (2)

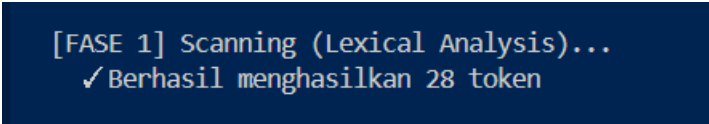
Gambar 3.10 dan 3.11 menunjukkan implementasi program utama yang berfungsi mengintegrasikan seluruh komponen compiler. Program utama

menjalankan proses kompilasi secara berurutan mulai dari Scanner, Parser, Semantic Analyzer, Intermediate Code Generator (ICG), hingga Optimizer untuk menghasilkan output akhir.

3.8 Pengujian Sistem

Pengujian sistem dilakukan untuk memastikan bahwa seluruh komponen compiler MiniPascal dapat bekerja sesuai dengan fungsinya. Pengujian dilakukan menggunakan source code MiniPascal yang memuat deklarasi variabel, operasi aritmatika, percabangan, perulangan, serta proses input dan output. Hasil pengujian diamati pada setiap fase kompilasi mulai dari Scanner, Parser, Semantic Analyzer, Intermediate Code Generation (ICG), hingga Optimizer. Hasil pengujian menunjukkan bahwa compiler berhasil memproses source code MiniPascal dan menghasilkan output yang sesuai pada setiap tahapan kompilasi.

3.8.1 Pengujian Scanner

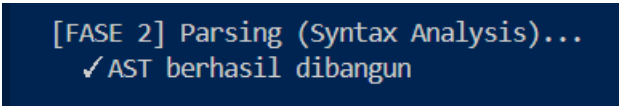
A screenshot of a terminal window with a dark blue background. It displays the text "[FASE 1] Scanning (Lexical Analysis)..." in a light blue font, followed by "✓ Berhasil menghasilkan 28 token" in a light green font.

```
[FASE 1] Scanning (Lexical Analysis)...  
✓ Berhasil menghasilkan 28 token
```

Gambar 3. 12 Pengujian Scanner

Pada tahap Scanner, compiler berhasil melakukan analisis leksikal terhadap source code MiniPascal dan menghasilkan 79 token. Token yang dihasilkan terdiri dari keyword, identifier, operator, literal, dan tanda baca yang digunakan sebagai masukan pada tahap parsing.

3.8.2 Pengujian Parser dan AST

A screenshot of a terminal window with a dark blue background. It displays the text "[FASE 2] Parsing (Syntax Analysis)..." in a light blue font, followed by "✓ AST berhasil dibangun" in a light green font.

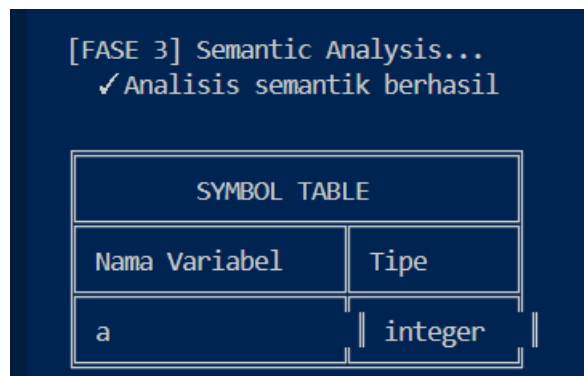
```
[FASE 2] Parsing (Syntax Analysis)...  
✓ AST berhasil dibangun
```

Gambar 3. 13 Pengujian Parser dan AST

Hasil pengujian menunjukkan bahwa Parser berhasil melakukan analisis sintaksis terhadap token yang dihasilkan Scanner. Selain itu, Parser juga berhasil

membangun Abstract Syntax Tree (AST) yang digunakan pada tahap analisis semantik.

3.8.3 Pengujian Semantic Analyzer



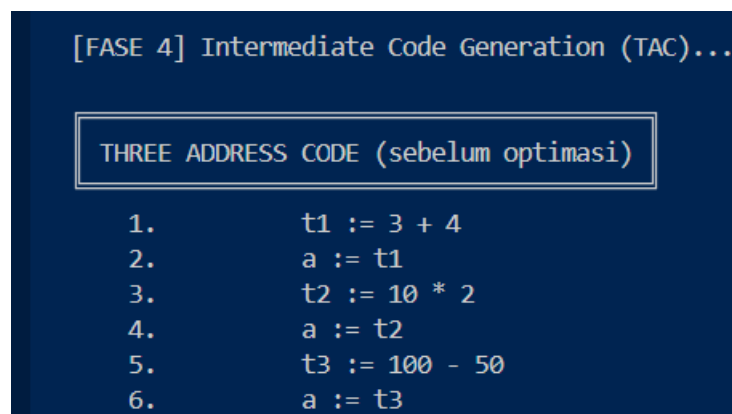
[FASE 3] Semantic Analysis...
✓ Analisis semantik berhasil

SYMBOL TABLE	
Nama Variabel	Tipe
a	integer

Gambar 3. 14 Pengujian Semantic Analyzer

Pada tahap Semantic Analyzer, compiler berhasil membangun Symbol Table dan memeriksa deklarasi variabel serta kesesuaian tipe data. Hasil pengujian menunjukkan bahwa seluruh variabel yang digunakan telah dideklarasikan dengan benar sehingga tidak ditemukan error semantik.

3.8.4 Pengujian Intermediate Code Generation (ICG)



[FASE 4] Intermediate Code Generation (TAC)...

THREE ADDRESS CODE (sebelum optimasi)	
1.	t1 := 3 + 4
2.	a := t1
3.	t2 := 10 * 2
4.	a := t2
5.	t3 := 100 - 50
6.	a := t3

Gambar 3. 15 Pengujian Intermediate Code Generation (ICG)

Compiler berhasil menghasilkan Three Address Code (TAC) dari Abstract Syntax Tree (AST). TAC digunakan sebagai representasi kode antara yang lebih sederhana sehingga memudahkan proses optimasi.

3.8.5 Pengujian Optimasi

```
[FASE 5] Optimasi Kode...

THREE ADDRESS CODE (setelah optimasi)

1.      t1 := 7
2.      a := t1
3.      t2 := 20
4.      a := t2
5.      t3 := 50
6.      a := t3

✓Optimasi selesai. Instruksi dieliminasi: 0
```

Gambar 3. 16 Pengujian Optimasi

Hasil pengujian menunjukkan bahwa Optimizer berhasil menerapkan teknik Constant Folding pada ekspresi yang seluruh operand-nya berupa konstanta. Sebagai contoh, operasi $3 + 4$ diubah menjadi 7, operasi $10 * 2$ diubah menjadi 20, dan operasi $100 - 50$ diubah menjadi 50.

3.8.6 Hasil Pengujian Keseluruhan

Berdasarkan hasil pengujian yang telah dilakukan, seluruh komponen compiler MiniPascal berhasil berjalan dengan baik. Scanner mampu menghasilkan token secara benar, Parser berhasil membangun AST, Semantic Analyzer mampu melakukan pemeriksaan semantik dan membangun Symbol Table, Intermediate Code Generator berhasil menghasilkan Three Address Code (TAC), serta Optimizer berhasil menerapkan teknik Constant Folding dan Dead Code Elimination. Dengan demikian, compiler MiniPascal yang dikembangkan telah memenuhi tujuan dan kebutuhan yang telah ditetapkan pada tahap perancangan sistem.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa compiler MiniPascal berhasil dikembangkan menggunakan bahasa pemrograman Python dengan menerapkan beberapa fase utama dalam teknik kompilasi. Compiler yang dibangun terdiri dari Scanner untuk analisis leksikal, Parser untuk analisis sintaksis dan pembentukan Abstract Syntax Tree (AST), Semantic Analyzer untuk pemeriksaan semantik dan pembangunan Symbol Table, Intermediate Code Generator (ICG) untuk menghasilkan Three Address Code (TAC), serta Optimizer yang menerapkan teknik Constant Folding dan Dead Code Elimination.

Hasil pengujian menunjukkan bahwa compiler mampu memproses source code MiniPascal dengan baik, menghasilkan token, AST, Symbol Table, TAC, serta melakukan optimasi terhadap kode antara yang dihasilkan. Dengan demikian, tujuan proyek untuk mengimplementasikan konsep dasar compiler pada bahasa MiniPascal telah berhasil dicapai sesuai dengan spesifikasi yang telah ditentukan.

4.2 Saran

Meskipun compiler MiniPascal yang dikembangkan telah mampu menjalankan fungsi dasar kompilasi, masih terdapat beberapa pengembangan yang dapat dilakukan pada penelitian atau proyek selanjutnya. Pengembangan tersebut antara lain menambahkan dukungan terhadap fitur bahasa yang lebih kompleks seperti array, procedure, function, dan record. Selain itu, compiler juga dapat dikembangkan dengan menambahkan tahap Code Generator sehingga mampu menghasilkan kode mesin atau kode target yang dapat dieksekusi secara langsung.

Pengembangan lain yang dapat dilakukan adalah meningkatkan kemampuan optimasi kode, memperbaiki mekanisme penanganan error, serta menambahkan antarmuka pengguna yang lebih interaktif sehingga compiler menjadi lebih mudah digunakan dan memiliki fitur yang lebih lengkap.

DAFTAR PUSTAKA

1. W. Adiyoso dan Y. A. Susetyo, “Pembangunan Compiler Domain Specific Language sebagai Generator Form HTML Menggunakan Python SLY,” *Jurnal Indonesia: Manajemen Informatika dan Komunikasi*, vol. 4, no. 2, pp. 449–461, 2023.
2. A. Apriansyah, T. I. Saputra, Fauziah, dan Y. F. Wijaya, “Desain Mesin Compiler untuk Penganalisa Leksikal, Sintaksis, Semantik, Kode Antara dan Error Handling Pada Bahasa Pemrograman Sederhana,” *Journal of Applied Informatics and Computing (JAIC)*, vol. 3, no. 1, pp. 1–7, 2019.
3. A. Baicoianu dan I. Plajer, “Considerations on Efficient Lexical Analysis in the Context of Compiler Design,” *Bulletin of the Transilvania University of Braşov: Mathematics and Computer Science*, vol. 3, no. 2, pp. 159–168, 2023.
4. E. A. Navas-López, “Modular and Didactic Compiler Design with XML Inter-Phases Communication,” *International Journal of Computer Science, Engineering and Information Technology (IJCSEIT)*, vol. 12, no. 1, pp. 1–20, 2022.
5. M. O. Odim, S. A. Arekete, B. O. Oguntunde, dan A. O. Lawal, “A Multi-Pass Compiler with Code Optimized Abstract Syntax Tree,” *International Journal of Software & Hardware Research in Engineering (IJSHRE)*, vol. 11, no. 5, pp. 31–39, 2023.
6. R. B. Barbosa, “Petit Programming Language and Compiler,” arXiv preprint arXiv:2311.14443, 2023.
7. Inggriani Liem, *Contoh Program Kecil dalam Bahasa Pascal*. Bandung: Institut Teknologi Bandung, 1999.
8. *Algoritma & Pemrograman: Teori dan Praktik dalam Pascal*, Edisi Kedua.